Detailed Paper Summaries + Concepts to Understand

PART 1: DETAILED PAPER SUMMARIES

Paper 1 — Achille, Rovere & Soatto (2019), "Critical Learning Periods in Deep Neural Networks"

Setup: The authors take a standard image-classification network and introduce a temporary "deficit" early in training — for example, blurring images (like a cataract) for the first portion of training, then switching to normal, sharp images for the rest. They vary exactly when this deficit starts, how long it lasts, and how deep/large the network is.

Core finding: If the deficit happens early and lasts long enough, the network never fully recovers — even though it eventually gets to train on totally normal data for the rest of its budget, its final accuracy stays permanently lower than a network that never had the deficit. If the same deficit happens later in training, the network recovers almost completely. Some deficits (like flipping images upside down, which doesn't damage low-level statistics) cause no lasting harm regardless of timing — showing it's specifically deficits that damage early low-level structure that cause permanent effects, not just "any disruption."

The mechanism (why this happens): The authors track something called Fisher Information (FI) — think of it as a measure of how strongly and confidently the weights between layers are "connected" or committed to the patterns they've learned. Counterintuitively, FI rises fast at the very start of training, then decreases and stabilizes — even though the model's performance keeps slowly improving. Their interpretation: early training is when the network is most "plastic" and open to reorganizing itself based on incoming data; once FI drops, the network has locked itself into a set of connections and loses the ability to substantially reorganize, even if better information arrives later. A deficit during the high-FI window gets "baked in" structurally; a deficit after FI drops doesn't, because the network wasn't going to reorganize around it anyway.

Bonus finding relevant to your embedding-reset intervention: the paper notes that this "forgetting" phase (where FI drops) might actually be necessary for building good, generalizable representations — and that standard pretrain-then-finetune transfer learning can sometimes hurt rather than help, because pretraining locks in connections around the wrong task before fine-tuning even starts. This is conceptually close to why deliberately resetting/forgetting things (like Chen et al.'s method) might restore useful flexibility.

Relevance to your study: This is your theoretical foundation. Your "Detect" phase is testing whether the same delayed-exposure-causes-permanent-cap pattern shows up for LLM skills, not just image statistics. Fisher Information is also a candidate diagnostic tool you could borrow for your own analysis.

Paper 2 — Kleinman, Achille & Soatto (2023 & 2024), two related papers

2a. Multisensory integration (CVPR 2023): Instead of a single input type (images), this paper studies networks that combine two correlated input streams (e.g., audio + video). They show that if the two streams are properly correlated early in training, the network learns to integrate them well; if that correlation is disrupted early on (even temporarily), the network permanently loses some of its ability to integrate the streams well, even after the correlation is restored. This shows critical periods aren't just about single-signal statistics — they also govern how a network learns to combine multiple sources of information, and the effect is tied to instability in the network's dynamics during the earliest part of training.

2b. Deep linear networks (2024): This is the more theoretical, stripped-down version. Linear networks have no nonlinear activation functions at all — they're about as simple as a neural network can be while still having multiple layers. The surprising finding: critical periods still show up here too. Since linear networks are simple enough to analyze mathematically, this suggests critical periods aren't a side-effect of complicated architectural choices (like attention, convolutions, or nonlinearities) — they're a more fundamental consequence of how gradient-based learning works over time, present even in the simplest possible learning system.

Relevance to your study: Together, these two papers broaden the claim from "this happens in vision CNNs" to "this is likely a general property of gradient-based learning itself." That strengthens the case that LLMs (a different architecture entirely) should plausibly show it too — which is exactly the premise your research is testing.

Paper 3 — Constantinescu, Pimentel, Cotterell & Warstadt (2024), "Do Language Models Have a Critical Period for Language Acquisition?"

Setup: The authors train language models on one language (L1), then introduce a second language (L2) starting at different delays — mirroring the human case of learning a first language early and a second language later in life (sometimes with reduced fluency).

Core finding: Unlike humans, the language models did NOT show a critical-period-like effect for L2 acquisition — delaying L2 exposure didn't cause a permanent cap on L2 performance the way delayed language exposure does in humans. The models eventually caught up regardless of when L2 started. The one condition where they DID see a human-like critical period effect was when they artificially reduced the model's plasticity using Elastic Weight Consolidation (EWC) — a regularization technique that penalizes changes to weights the model considers "important" for what it already knows, deliberately making parts of the network resistant to further change.

What this means: Under normal training, LLMs seem to remain "plastic" (able to reorganize and learn new things) far longer than biological brains do — they don't naturally develop the same kind of learning-window closure. The human-like effect only emerges when you deliberately damage the model's ability to keep learning.

Important nuance for your paper: this null result is specifically about cross-lingual (whole new language) transfer, tested at the level of overall L2 proficiency. It does NOT directly test whether a specific skill (arithmetic, one syntax construction) introduced late within a single language's pretraining behaves the same way. Those are different granularities and different kinds of "newness" (new language vs. new skill within a familiar language) — which is exactly the gap your research fills.

Relevance to your study: This is the paper whose null result you're directly engaging with. Your Related Work section needs to clearly explain why your setup (skill-specific, within-language, natural pretraining without artificial EWC damage) is different enough that a different result wouldn't contradict theirs — it would extend the boundary of when critical periods do and don't appear in LLMs.

Paper 4 — Chen et al. (2023), "Improving Language Plasticity via Pretraining with Active Forgetting"

Setup: The authors modify the standard pretraining recipe by periodically resetting the model's embedding layer (the lookup table converting tokens to vectors) back to fresh random values at regular intervals throughout pretraining — not once, but repeatedly, as a built-in habit of the training process. The rest of the network (all the transformer layers) keeps its weights and keeps learning normally; only the embeddings get periodically wiped and relearned.

Core finding: Models trained this way become much better at quickly learning new languages later — when you take the finished model and try to adapt it to an entirely new language it never saw during pretraining, it adapts faster and performs better in low-data conditions, especially for languages very different from English (different scripts, word orders, etc.), compared to a model trained normally without this periodic-reset habit.

Why this works (their explanation): By repeatedly forcing the model to relearn its embeddings from scratch throughout training, the model essentially gets "practice" at rapidly building good token representations from nothing, rather than becoming overly dependent on one fixed, specific embedding space. This builds a general capacity for fast relearning — similar in spirit to meta-learning ("learning how to learn quickly") — rather than a specific piece of knowledge.

Important nuance for your paper: this is a standing training policy applied throughout pretraining from the start, not a one-time rescue applied after a specific skill's window has already closed. It's also tested on whole-new-language transfer after pretraining is fully finished, not on a skill embedded within a single language's pretraining data. Your use of "embedding reset" as intervention #1 borrows the mechanism but applies it completely differently: as a single, targeted, retroactive fix at one specific point, for one specific missed skill, compared against three other intervention types.

Relevance to your study: This is the direct methodological ancestor of your most important intervention (embedding/layer reset). You're testing whether a mechanism that works as

an "all-along" policy for whole-language transfer also works as a "one-time rescue" for a specific missed skill — a meaningfully different and harder question.

Paper 5 — "Curriculum Learning for LLM Pretraining: An Analysis of Learning Dynamics" (2026)

Setup: The authors train a whole suite of models (14M to 410M parameters, plus one 1B model) on 300 billion tokens each, using the Pile dataset, comparing four different data-ordering strategies: plain random shuffling, and three "curricula" that sort training examples from easier to harder using different criteria — Age-of-Acquisition (words children learn earlier come first), Word Frequency (common words first), and Verb Variation (VV, sentences with simpler verb structure first).

Core finding on training dynamics: Regardless of which ordering is used, all models pass through the same general sequence of underlying "phases" during training (identified using Hidden Markov Model analysis of training trajectories) — the curricula don't create new phases, they just change how much time the model spends in each existing phase.

Core finding on smaller models specifically: In smaller models (up to about 160M parameters), random ordering produces noisier gradient signals and shows signs of the output layer "saturating" (losing useful capacity) late in training — both of which are linked to lower final accuracy compared to structured curricula. In other words, for small models, the order you feed data in has a real, measurable effect on both training stability and final performance — an effect that shrinks and mostly disappears in larger models.

Skill-specific finding (most relevant to you): at the level of individual linguistic capabilities, specific curricula help or hurt specific skills unevenly — the Verb Variation curriculum specifically improved performance on wh-questions and causative constructions, while the Word Frequency curriculum could actually underperform Random on certain other constructions (like superlative quantifiers). This means ordering effects aren't uniform across all of language — they're construction-specific, which is exactly the granularity your research needs.

Relevance to your study: This paper gives you two things directly. First, hard evidence that data ordering has real, measurable effects on smaller models (your exact scale) without needing any artificial plasticity damage — directly countering the implication of paper 3's null result that LLMs are simply immune to ordering effects. Second, it specifically flags wh-questions as a construction sensitive to data ordering, which is exactly why your syntax skill choice (wh-question object-gap extraction) isn't arbitrary — it's the one construction already shown to respond to timing/ordering in prior work at your model scale.

HOW THE 5 PAPERS FIT TOGETHER

- Papers 1-2 establish critical periods as a general phenomenon in gradient-based learning systems (vision, multisensory, even simple linear networks) — not a fluke of

one architecture.

- Paper 3 tests the most direct prior version of your question in LLMs specifically, and finds a null result for cross-lingual transfer — this is the result you need to explain around.
- Paper 4 gives you your leading intervention mechanism, tested in a different context (standing policy, whole-language) than your use of it (one-time rescue, specific skill).
- Paper 5 gives you direct evidence that ordering effects ARE real and measurable in LLMs at your scale, specifically on your chosen syntax construction — creating a genuine tension with paper 3 that your study is positioned to help resolve.

PART 2: CONCEPTS YOU NEED TO UNDERSTAND SMOOTHLY

These are additional to the ML basics primer (model/embeddings/attention/layers/checkpoints/perplexity) already covered — this list is specific to reading these 5 papers and running your analysis.

Fisher Information (FI) / Fisher Information Matrix (FIM)

A mathematical measure (originally from statistics, dating back to Fisher 1925) of how sensitive a model's output is to small changes in its parameters — practically, how much information the current weights encode about the training data, and how "committed" or confidently-connected the layers are. In these papers, FI rising then falling is used as evidence for an early "plastic" phase followed by a "consolidation" phase. You don't need to derive it yourself, but you should understand it as a proxy for "how much can this network still be reshaped by new information right now."

Elastic Weight Consolidation (EWC)

A training technique originally designed to prevent catastrophic forgetting: it identifies which weights are "important" for a task the model already knows (using something like Fisher Information again) and adds a penalty discouraging those weights from changing much during further training. In paper 3, applying EWC to a language model artificially recreates a human-like critical period — this is a good illustration of critical periods being connected to plasticity loss, which EWC induces on purpose.

Gradient Noise Scale (GNS)

A measure of how noisy/inconsistent the gradient signal is across different mini-batches during training. High GNS generally means the optimizer is getting conflicting signals about which direction to update weights, leading to less stable training. Paper 5 uses this to show random-ordering training is noisier than curriculum-ordered training in small models — worth understanding since you might use a similar diagnostic in your own analysis.

Hidden Markov Model (HMM) latent phase analysis

A statistical technique for finding hidden ("latent") states that a system moves through over time, based only on observable signals. Paper 5 fits an HMM to training trajectories to identify a shared sequence of training "phases" across different data orderings. You likely won't need to implement this yourself, but understanding the concept helps you interpret why they claim curricula don't create new phases, just reallocate time within them.

BLiMP (Benchmark of Linguistic Minimal Pairs)

A standard evaluation suite for testing whether a language model prefers grammatically correct sentences over minimally-different incorrect ones (e.g., "the cat that the dog chased was tired" vs. an ungrammatical variant). This is how you'll measure your syntax skill's accuracy — the model is shown pairs of sentences and scored on whether it assigns higher probability to the grammatical one.

Catastrophic forgetting

A general phenomenon in neural networks where learning something new causes the network to lose previously-learned capabilities, because gradient updates for the new task overwrite weights that were important for the old one. This is the exact risk flagged in your critical pitfalls table — recovering a delayed skill by damaging general language ability is catastrophic forgetting wearing a disguise, not genuine reopening.

Activation patching / causal tracing

An interpretability technique: you run the model normally, but at one chosen internal point (a specific layer, or specific attention head), you replace ("patch") its actual computed values with values taken from a different run (e.g., from the baseline model, or from a different input). If patching that one location changes the output to match the source run, that location is causally responsible for the behavior in question. This is your main tool for the Diagnose phase — testing whether the recovered model relies on the same components the original baseline did.

Ablation

Deliberately disabling or zeroing out part of a network (a whole layer, a specific attention head, a specific weight subset) and observing how much performance drops as a result. Large performance drop = that part was doing important work for the task; small/no drop = that part wasn't critical, or something else compensated.

CKA (Centered Kernel Alignment) / cosine similarity

Two different mathematical ways to measure how similar two sets of internal representations are, even when the raw numbers don't match up directly (e.g. comparing hidden states from two differently-trained models). Cosine similarity is simpler (just measures the angle between vectors); CKA is a more robust technique commonly used specifically to compare representations across different neural networks. You'll use one or

both of these in the Diagnose phase to check whether the recovered model's internal representations resemble the original baseline's.

Pruning-based localization

Finding the smallest possible subset of a network's weights/components that's still sufficient to perform a task well, usually by iteratively removing (pruning) the least important-seeming parts and checking if performance holds. Comparing the "pruned footprint" of the original baseline vs. the recovered model tells you whether they rely on overlapping or entirely different parts of the network.

Mixed-effects regression

A statistical model used when your data has both "fixed" effects you care about (like delay_onset, model_size — your actual experimental variables) and "random" effects that add noise you need to account for but don't care about directly (like random seed, or which specific run). This lets you ask "does delay_onset really predict accuracy" while properly accounting for run-to-run variation, rather than treating every run as fully independent.

Held-out data

Data the model never saw during training, used purely for evaluation. Essential for every metric in this study (skill accuracy, general perplexity) — if you evaluate on data the model was trained on, you're measuring memorization, not real capability.

Bottom Line

You now have: (1) what each paper actually found and why it matters to your study specifically, and (2) the vocabulary needed to understand your own eval harness, interventions, and especially the Week 8 circuit-diagnosis phase without those terms feeling foreign. Combine this with the earlier ML primer (covering model architecture basics) and you should be able to follow — and meaningfully sanity-check — everything Fable 5/Claude Code builds for you.